

```

/*
 * uart.c
 *
 * Created: 14-09-2026 20:38:06
 * Author: ARJUN
 */

void uart_init()
{
    // step 1: Baud rate calculation for 9600
    /*
    F_cpu = 16,000,000
    Baud = 9600

    value = (F_cpu / (16 x baud rate)) - 1
            = (16,000,000 / (16 x 9600)) - 1
            = 104.16 - 1
            = 103.16 -> 103
    */

    UBRR0H = 0;
    UBRR0L = 103;

    // step 2: Enabling Transmitter and receiver
    UCSRB |= ((1 << RXEN0) | (1 << TXEN0));

    // step 3: frame format configuration
    UCSRC |= ((1 << UCSZ00) | (1 << UCSZ01));
    UCSRB &= ~(1 << UCSZ02);

    // choosing parity: no parity
    UCSRC &= ~(1 << UPM01) | (UPM00));

    // stop bit selecting: 1 stop bit
    UCSRC &= ~(1 << USBS0);

    // step 4:
}

void uart_transmit(char data)
{
    // waiting until the buffer is ready
    while (!(UCSR0A & (1 << UDRE0)));

    // giving data to the buffer once it is ready
    UDR0 = data;
}

/*
 * uart_transmit.c
 *
 * Created: 14-09-2026 20:07:21
 * Author : ARJUN
 */

```

*/

```
#define F_CPU 16000000UL
```

```
#include <avr/io.h>
```

```
#include <util/delay.h>
```

```
#include "uart.h"
```

```
int main(void)
```

```
{
```

```
    uart_init();
```

```
    char data = 'H';
```

```
    while (1)
```

```
    {
```

```
        uart_transmit(data);
```

```
        uart_transmit('\n');
```

```
    }
```

```
}
```

```
#ifndef UART_H_
```

```
#define UART_H_
```

```
void uart_init(void);
```

```
void uart_transmit(char data);
```

```
#endif
```

Adc

adc.h

```
#ifndef ADC_H_
```

```
#define ADC_H_
```

```
#include <stdint.h>
```

```
void adc_init(void);
```

```
uint16_t adc_read(uint8_t channel);
```

```
#endif /* ADC_H_ */
```

adc.c

```
#define F_CPU 16000000UL
```

```
#include <avr/io.h>
```

```
#include "adc.h"
```

```
void adc_init(void)
```

```
{
```

```
    ADMUX |= (1 << REFS0); // Set Aref = AVcc = 5V
```

```
    ADCSRA |= (1 << ADEN) |
```

```
        (1 << ADPS0) |
```

```

        (1 << ADPS1) |
        (1 << ADPS2); // ADC Enabled. Prescaler set to 128
    }

uint16_t adc_read(uint8_t channel)
{
    channel &= 0x07; // Channel limited to 7;

    ADMUX = (ADMUX & 0xF8) | channel; // Selected channel without affecting other bits of
    ADMUX

    ADCSRA |= (1 << ADSC); // start conversion

    while (ADCSRA & (1 << ADSC)); // wait until conversion

    return (ADC);
}

```

Main.c

```

#define F_CPU 16000000UL

#include <avr/io.h>
#include <stdint.h>
#include "adc.h"

int main(void)
{
    uint16_t adc_value;
    uint16_t temperature;

    adc_init();

    while (1)
    {
        // Read LM35 connected to ADC0
        adc_value = adc_read(0);

        // Convert ADC value to temperature
        // Vref = 5V
        // 10-bit ADC = 0 to 1023
        // LM35 = 10mV per °C
        temperature = (adc_value * 500UL) / 1023;

        // Use 'temperature' here
        // Example: send it through UART or display on LCD
    }

    return 0;}

```

```

/*
 * lcd.c
 *
 * Created: 11-09-2026 19:44:21
 * Author: ARJUN
 */

#include "lcd.h"

// lets start with that enable pulse
static void lcd_enable_pulse(void)
{
    LCD_EN_PORT |= (1 << LCD_EN_PIN);           // enable pin =1
    _delay_us(1);                               // these delay values come from datasheet
    LCD_EN_PORT &= ~(1 << LCD_EN_PIN);          // enable pin =0
    _delay_us(50);                              // allow LCD
to latch
}

// this function sends a nibble to the data pins (b3..b0 -> D7..D4)
static void lcd_send_nibble(uint8_t nib)
{
    // Clear PD2..PD5 by our mapping
    LCD_D_PORT &=
~((1<<LCD_D7_PIN)|(1<<LCD_D6_PIN)|(1<<LCD_D5_PIN)|(1<<LCD_D4_PIN));
// Making all data pins 0 initially

    // now set the pins according to the data/command bit weights
    if (nib & (1 << 3))
        LCD_D_PORT |= (1 << LCD_D7_PIN); // Bit 3 -> D7 (PD2)
    if (nib & (1 << 2)) LCD_D_PORT |= (1 << LCD_D6_PIN); // Bit 2 -> D6
(PD3)
    if (nib & (1 << 1)) LCD_D_PORT |= (1 << LCD_D5_PIN); // Bit 1 -> D5
(PD4)
    if (nib & (1 << 0)) LCD_D_PORT |= (1 << LCD_D4_PIN); // Bit 0 -> D4
(PD5)

    */

    lcd_enable_pulse(); //calling the enable fuction
}

// this function allows us to send a byte
static void lcd_send_byte(uint8_t value, uint8_t rs) // here we receive data and rs
value
{

```

```

        if(rs==1)
        {
            LCD_RS_PORT |= (1<<LCD_RS_PIN);           // IT'S A Data.
SO RS pin =1
        }
        else
        {
            LCD_RS_PORT &= ~(1<<LCD_RS_PIN);           // it's a
Command. So RS pin0
        }
        lcd_send_nibble(value >> 4);                 // sending upper nibble 0111 0011 &
        lcd_send_nibble(value & 0x0F);                 // sending lower nibble    0000 1111
                                                    -----
                                                    0000 0011
    }

```

// this function sends commands (instructions)

```
void lcd_cmd(uint8_t c)
```

```

{
    lcd_send_byte(c, 0);
    // Clear (0x01) and Return Home (0x02) need longer delays in write-only mode
    if (c == 0x01 || c == 0x02)
        _delay_ms(2);
}

```

// This function allows us to send data

```
void lcd_data(uint8_t d)
```

```

{
    lcd_send_byte(d, 1);
}

```

// this function allows us to set the cursor

```
void lcd_set_cursor(uint8_t row, uint8_t col)
```

// receives row and column

value 2X16 LCD

```

{
    uint8_t addr = (row ? 0x40 : 0x00) + (col & 0x0F); // this is a ternary
operator.
    lcd_cmd(0x80 | addr);                               // send (80 + addr) as a
command.
}

```

//This function allows us to print a string

```

void lcd_print(const char *s)
{
    while (*s)          // every string ends with '\0'. This command checks
    whether *s=\0
    {
        lcd_data((uint8_t)*s++);
    }
}

// this function allows us to clear the lcd
void lcd_clear(void)
{
    lcd_cmd(0x01);
}

// this function allows us to return the cursor to home
void lcd_home(void)
{
    lcd_cmd(0x02);
}

// this function initialises the lcd. (initialisation sequence from datasheet)

void lcd_init(void)
{
    // Make control and data pins outputs
    LCD_RS_DDR |= (1<<LCD_RS_PIN);          // Setting data direction
registers for RS and EN pin
    LCD_EN_DDR |= (1<<LCD_EN_PIN);
    LCD_D_DDR |=
(1<<LCD_D7_PIN)|(1<<LCD_D6_PIN)|(1<<LCD_D5_PIN)|(1<<LCD_D4_PIN); //
setting DDR for data pins

    _delay_ms(20);          // power-up wait
    LCD_RS_PORT &= ~(1<<LCD_RS_PIN);          // RS=0

    // 8-bit wake-up sequence (sent as high nibbles). why nibbles?
    LCD_EN_PORT &= ~(1 << LCD_EN_PIN);          // make EN =0 .
    lcd_send_nibble(0x03);
    _delay_ms(5);
    lcd_send_nibble(0x03);
    _delay_us(150);
    lcd_send_nibble(0x03);
    _delay_us(150);

    //Switch to 4-bit

```

```

    lcd_send_nibble(0x02);
    _delay_us(150);

    // note that from here onwards its cmd !

    //Function set: 4-bit, 2 lines, 5x8 font
    lcd_cmd(0x28);

    //Display off

    lcd_cmd(0x08);

    //Clear
    lcd_cmd(0x01);

    //Entry mode: increment, no shift
    lcd_cmd(0x06);

    // Display on, cursor off, blink off
    lcd_cmd(0x0C);
}

// function to print integers. (We use recursion)
void lcd_print_uint16(uint16_t v)                                // lcd_print_uint1(1432);
{
    if (v >= 10)
    {
        lcd_print_uint16(v / 10);                                // print higher
        digits first
    }
    lcd_data('0' + (v % 10));                                     // then print the last
    digit
}

/*
 * lcd.h
 *
 * Created: 11-09-2026 19:44:32
 * Author: ARJUN
 */

/*
 * lcd.h
 *

```

```

* Created: 09-09-2026 17:45:49
* Author: ARJUN
*/
#ifndef LCD_H_
#define LCD_H_

#include <avr/io.h>
#include <util/delay.h>
#include <stdint.h>

// rs = 12, en = 11, d4 = 5, d5 = 4, d6 = 3, d7 = 2

// RS pin
#define LCD_RS_PORT PORTB
#define LCD_RS_DDR DDRB
#define LCD_RS_PIN PB4

// Enable pin
#define LCD_EN_PORT PORTB
#define LCD_EN_DDR DDRB
#define LCD_EN_PIN PB3

// Data pins
#define LCD_D_PORT PORTD
#define LCD_D_DDR DDRD

#define LCD_D7_PIN PD2
#define LCD_D6_PIN PD3
#define LCD_D5_PIN PD4
#define LCD_D4_PIN PD5

// Public functions

void lcd_init(void);
void lcd_cmd(uint8_t c);
void lcd_data(uint8_t d);
void lcd_set_cursor(uint8_t row, uint8_t col);
void lcd_print(const char *s);
void lcd_clear(void);
void lcd_home(void);
void lcd_print_uint16(uint16_t v);

#endif /* LCD_H_ */
/*
* Parallel_port_programming_16x2_lcd.c

```

```

*
* Created: 11-09-2026 19:43:50
* Author : ARJUN
*/

/*
* 16x3_lcd_interface.c
*
* Created: 09-09-2026 17:45:13
* Author : ARJUN
*/

#define F_CPU 16000000UL

#include <avr/io.h>
#include <util/delay.h>
#include "lcd.h"

int main(void)
{
    lcd_init();
    lcd_set_cursor(0, 0);
    lcd_print("Hello LCD");

    lcd_set_cursor(1, 0);
    lcd_print("It works! ");

    while (1)
    {
        lcd_set_cursor(0, 0);
        lcd_print("Hello LCD");

        lcd_set_cursor(1, 0);
        lcd_print("It works! ");
    }
}

```